

DNS Resolver Recommender

Intermediate Results: Measuring DNS over HTTPS in the Browser

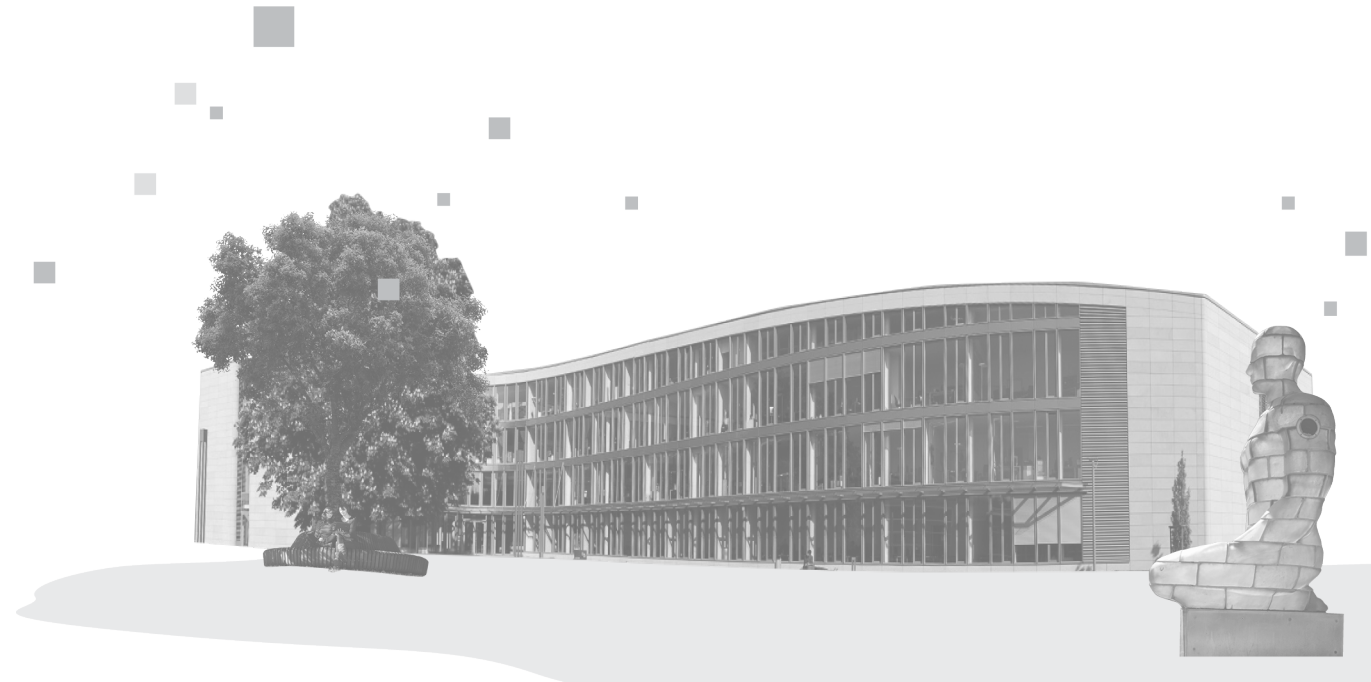
Jhannes Ernesto Reimann

Course: Trends in Internet Measurements

Supervisor: Robert Richter

**Design IT.
Create Knowledge.**

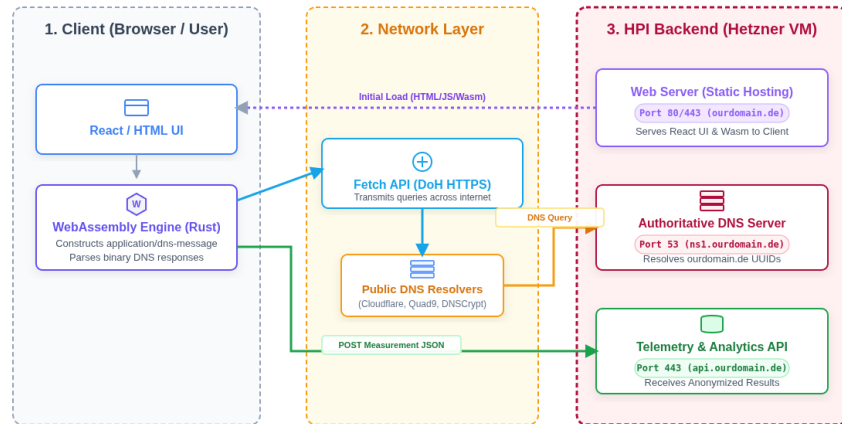
www.hpi.de



Recap: From Proposal to Production



Planned (proposal):

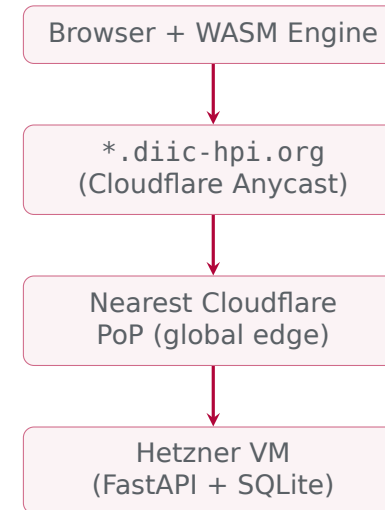


Own authoritative DNS on Hetzner, plus 30s Anycast record rotation.

Why Cloudflare instead:

- Self-hosted DNS measures only the path to Germany
- Anycast routes each user to the nearest PoP globally

Built:



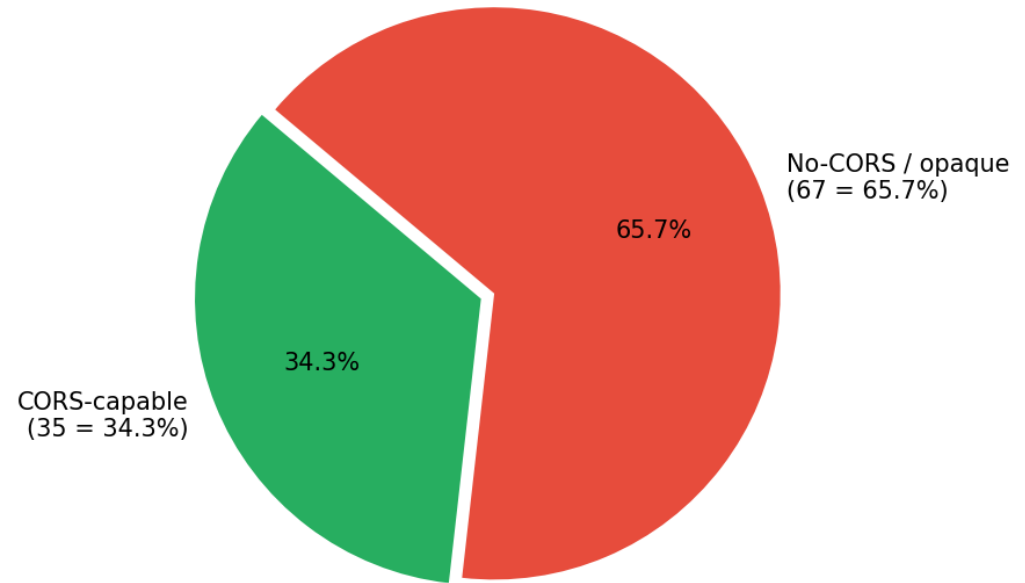
Wildcard DNS, WASM, dynamic CORS, two-tier verification.

The 102 Public DoH Resolvers



- DoH encrypts DNS in HTTPS: no ISP surveillance, no manipulation of responses in transit
- 102 DoH endpoints from DNSCrypt v3.28
- VLP decoding (0x80 continuation bit) unlocked Google, Quad9, AdGuard, NextDNS, Yandex
- 35 CORS (34.3%), 67 opaque (65.7%), 0 permanently dead
- DNSSEC 84.3%, no-logs 84.3%, unfiltered 43.1%

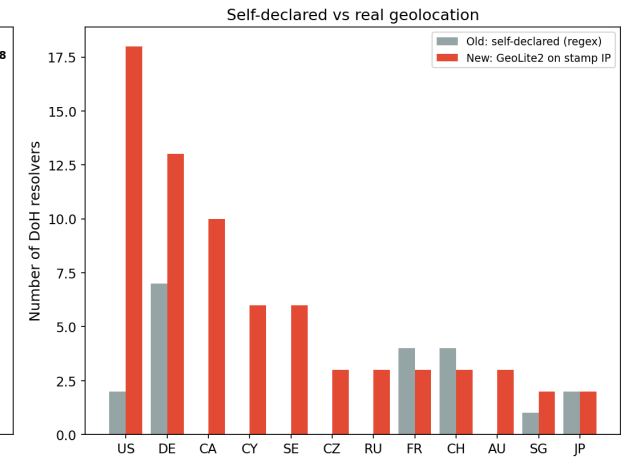
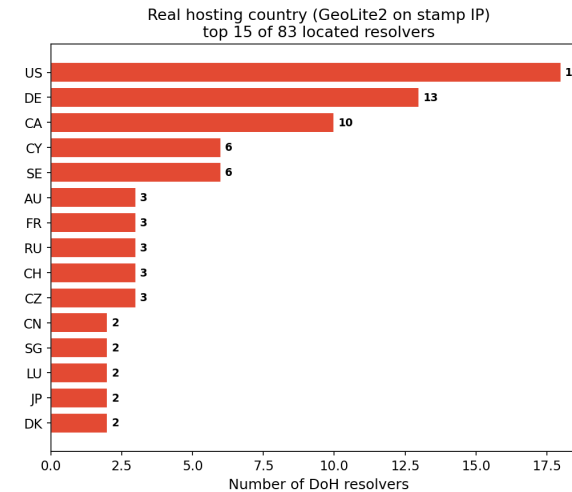
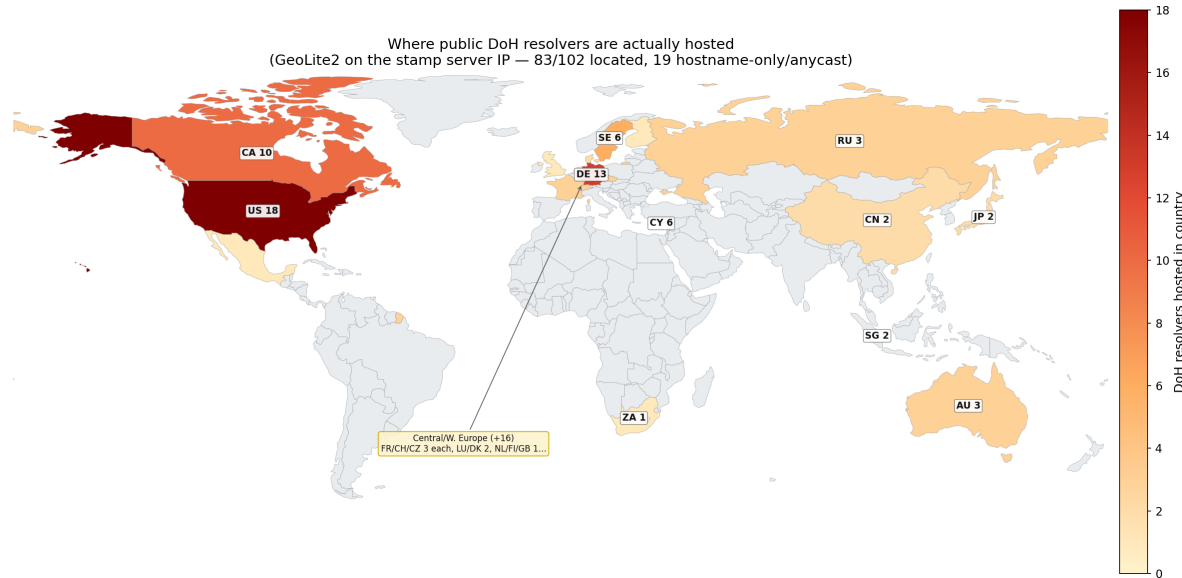
Browser CORS support among public DoH resolvers
(real telemetry, 102 resolvers, 203 runs)



Per run a German client reaches ~29 CORS resolvers;
~4/102 time out per DE run (4%), but 0 are permanently dead
(every resolver answered in ≥ 1 run).

[3] DNSCrypt (2026). [6] IETF (2024).

Where Resolvers Are Actually Hosted



- GeoLite2 on decoded stamp IPs (83/102): **US 18, DE 13, CA 10, CY 6, SE 6**
- US-dominated due to anycast/cloud hosting (Cloudflare, Google, Quad9, NextDNS)
- Canada (10), Cyprus (6), Sweden (6) invisible in community self-declared metadata

MaxMind GeoLite2-Country database

The CORS Gap: Why 65.7% of DoH Resolvers Cannot Serve the Browser



RFC 8484 (Hoffman & McManus, 2018), Section 1:

"[...] allowing web applications to access DNS information via existing browser APIs in a safe way consistent with Cross Origin Resource Sharing (CORS)."

DoHSpeedTest (script.js, lines 474-488):

```
if (allowCors) {
  response = await fetch(dohUrl, {mode: 'cors'});
} else {
  response = await fetch(urlWithParam,
    {method: 'GET', mode: 'no-cors'});
}
// opaque: status=0, headers={}, body=null
allowCors: true hardcoded for Cloudflare, DNS.SB, DNSPod, Google. All others in opaque mode.
```

- RFC 8484 cites CORS as a design goal but has **no normative requirement**
- Browser Fetch API: without Access-Control-Allow-Origin, JavaScript cannot read the response
- 65.7% lack CORS. A cheating resolver returning empty HTTP 200 in 2ms would rank first
- **First systematic measurement** of this gap

[5] Hoffman & McManus (2018), RFC 8484. [14] WhatWG (2024), Fetch Standard.

Two-Tier Verification: Proving a Resolver Did the Work



Tier 1 - CORS Auto-Verification

- WASM validates DNS wire format via hickory-proto
- Response code = NOERROR, answer section non-empty
- Proof of recursive resolution (code on slide 10)
- Immediate, 35 resolvers, no external API

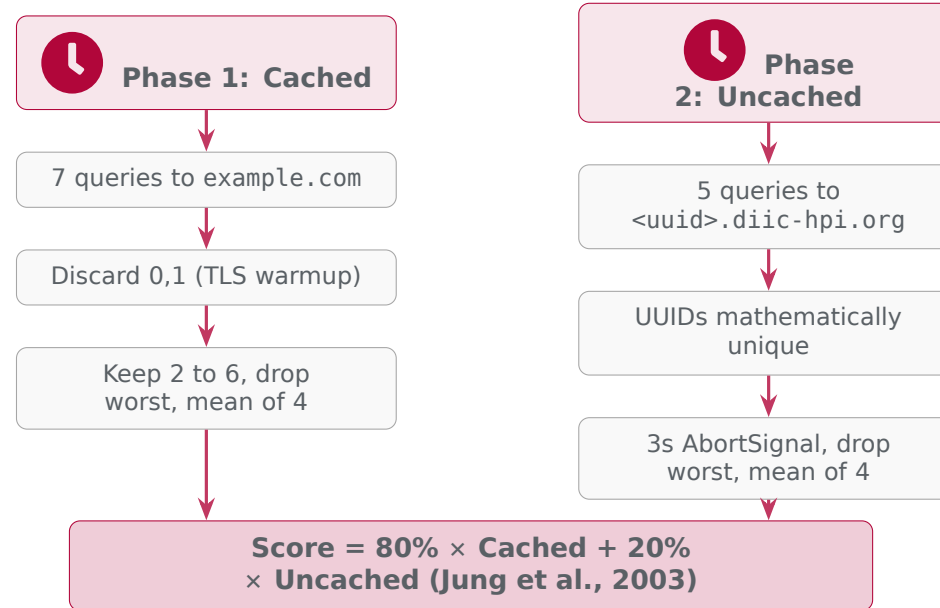


Tier 2 - Authoritative Log Verification

- Cloudflare GraphQL (dnsAnalyticsAdaptive)
- Exponential backoff: 10s, 8s, 16s, 32s (90s budget)
- Verifies UUID appeared in authoritative logs
- 67 resolvers, background polling

[2] Cloudflare (2026). Analytics GraphQL API, Sampling.

Measurement Design



[8] Jung, Sit, Balakrishnan, & Morris (2003). IEEE/ACM Trans. Networking, 10(5).

What One Measurement Run Stores



runs table (1 row)

- Timestamp (Europe/Berlin), ASN + org, country code
- Browser family & major version, OS family, browser language
- Aggregate counts: total, CORS, opaque, dead, verified, paradox
- Avg cached & uncached latency across all resolvers



Privacy by design

- **No raw IPs:** GeoLite2 ASN/country lookup in-memory, IP discarded before any data hits disk
- **No raw User-Agent:** parsed to coarse browser/OS family, raw string never stored



measurements table (102 rows)

- Resolver ID, name, URL
- Cached avg ms, uncached avg ms, weighted score ms
- **Raw 5-query timing arrays** (cached_times, uncached_times) as JSON
- CORS, DNSSEC, no-logs, no-filter flags
- Self-declared resolver country, verification status, paradox flag



Why raw arrays?

- Enables estimator replay (arithmetic vs. trimmed mean) on identical data
- $203 \text{ runs} \times 102 \text{ resolvers} \times 10 \text{ queries} = \sim 206\text{K timing samples}$

Live at dns.diic-hpi.org – Run It on Your Device



- 102 DoH resolvers from your browser in ~60s
- Scan QR code, log in with **dnsrr** / **dnsrr**



Resolver	Cached (ms)	Uncached (ms)	Score (ms)	Status
artikel10-doh-ipv4 FASTEST Setup [DNSSEC No-Logs Unfiltered] DNSSEC, No-filter and No-log DoH resolver operated by Artikel10 association. Homepage: https://artikel10.org Browser (DoH URL): https://dns.artikel10.org/dns-query Copy System DNS IP: 217.197.91.153 Copy CORS Status: ✓ Supports CORS (fully verified browser queries) Enter the DoH URL in your browser's Secure DNS settings (e.g. Chrome/Firefox Settings -> Secure DNS) or use the IP in your system network settings.	7.2 ms [6.9, 6.9, 8.0, 8.8, 6.8]	10.5 ms [20.8, 11.0, 10.5, 10.9, 9.7]	7.8 ms	✓ Verified (DNS) (CORS response parsed)
quad9-doh-ip4-port443-nofilter-ecs-pri Setup [No-Logs Unfiltered]	8.0 ms [8.1, 7.7, 7.8, 8.3, 8.4]	12.8 ms [15.8, 10.9, 19.4, 12.0, 12.6]	8.9 ms	🔍 Verifying... (No-CORS)
quad9-doh-ip4-port443-filter-ecs-pri Setup [DNSSEC No-Logs]	8.5 ms [11.6, 8.9, 8.6, 8.7, 7.9]	16.3 ms [29.7, 32.2, 11.9, 11.7, 11.8]	10.1 ms	🔍 Verifying... (No-CORS)
nextdns-ultralow Setup [DNSSEC No-Logs Unfiltered]	9.4 ms [8.6, 9.7, 8.7, 12.2, 10.6]	17.5 ms [31.7, 12.0, 32.0, 12.8, 13.3]	11.0 ms	🔍 Verifying... (No-CORS)
quad9-doh-ip6-port443-filter-pri Setup	10.9 ms [10.4, 11.2, 10.9, 11.2]	14.1 ms [25.7, 13.7, 15.6, 13.1]	11.6 ms	🔍 Verifying...

WASM vs. JavaScript: Correctness Over Bundle Size



src/wasm/src/lib.rs – DNS response validation:

```
let bytes =
    Uint8Array::new(&array_buffer)
        .to_vec();
let msg =
    Message::from_vec(&bytes)?;

if msg.header()
    .get_response_code()
    != Rcode::NOERROR {
    return Err(...);
}
if msg.answers().is_empty() {
    return Err(...);
}
```

Four lines of Rust invoke ~2,000 lines of hickory-proto, battle-tested in production DNS servers.

- 148 lines Rust, bundle **≈ 110 KB gzipped**
- Handles: label compression pointers, EDNS0, DNSSEC RRSIG, truncated responses
- Pure JS (dns-packet, ~8 KB) risks silent parse failures on edge cases
- For a research tool: correctness > bundle size

Six Things No Other Browser-Based DoH Benchmark Does



Dynamic CORS probing at runtime (no hardcoded lists)



DNS wire-format validation via WASM engine



CORS gap: first systematic measurement (34.3% / 65.7%)



Authoritative log verification for opaque resolvers



Real GeoLite2 geolocation of 83 resolver IPs

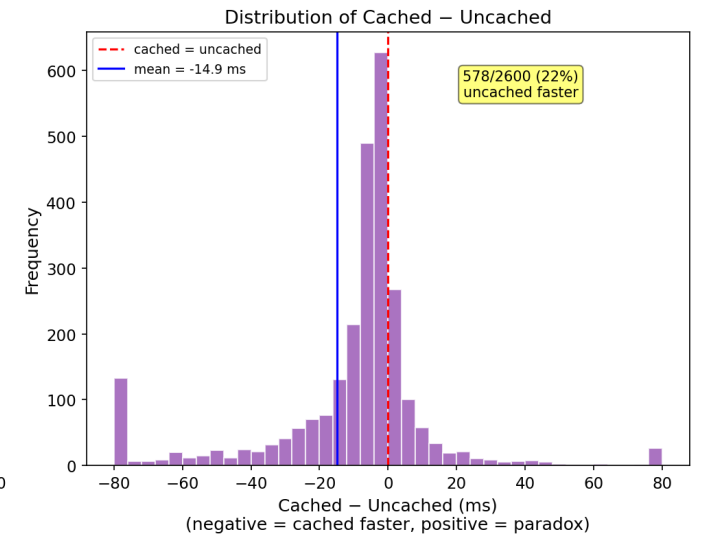
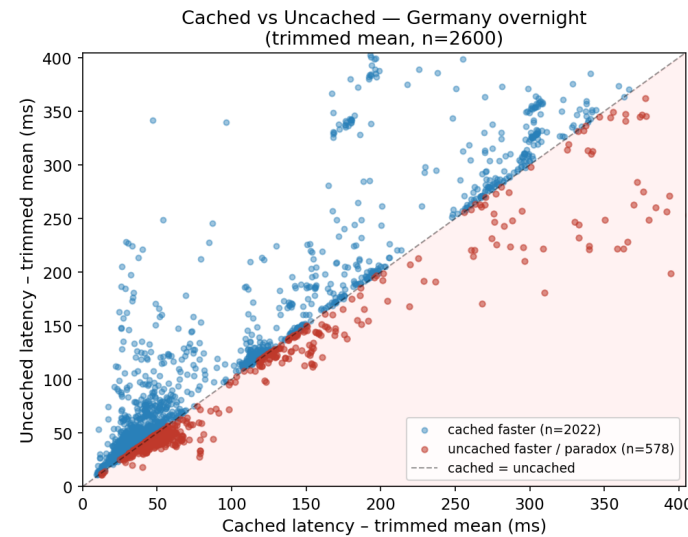


Multi-location campaign: 51 runs, 6 countries

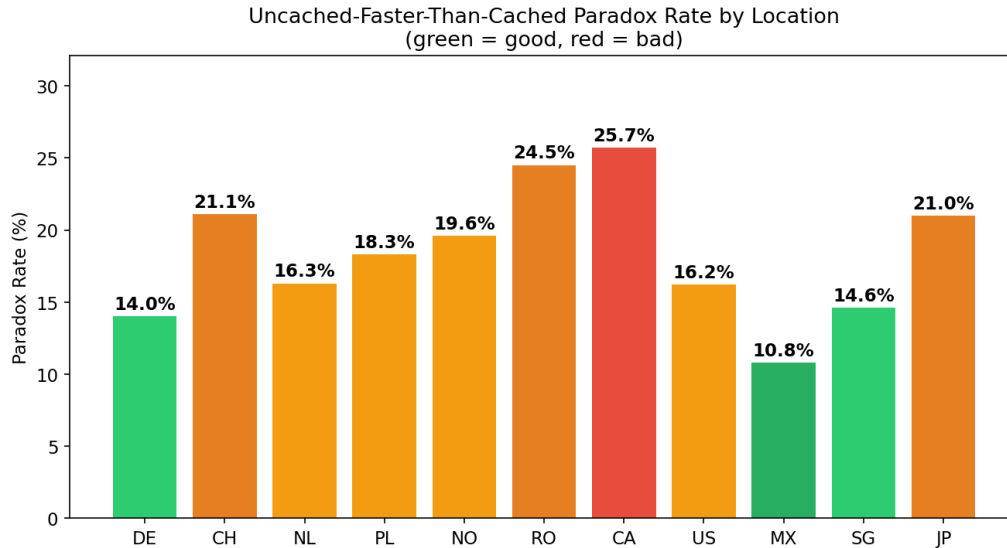
The Cached-vs.-Uncached Paradox



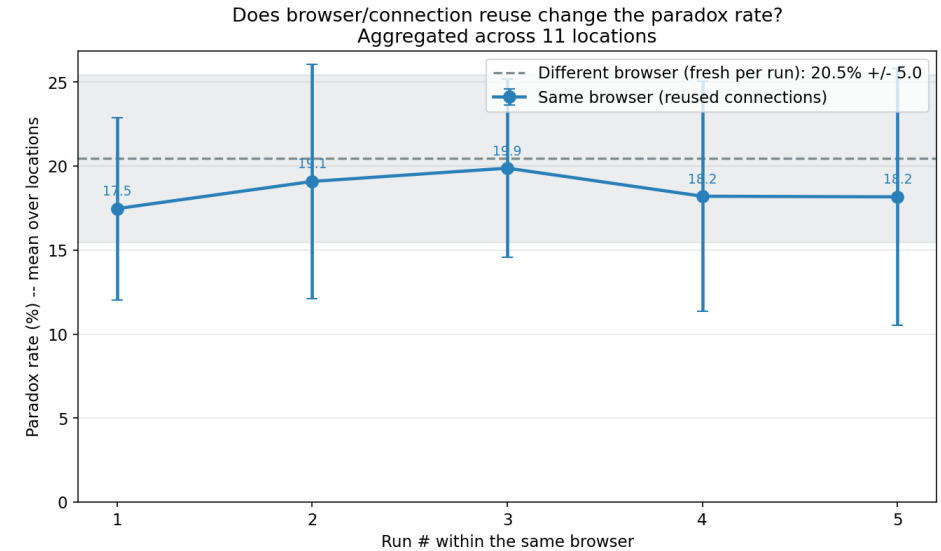
- Uncached latency can be lower than cached. For fast resolvers with a narrow gap, per-query jitter flips ~22% of comparisons
- Root cause: two phases hit different authoritative infrastructures. When gap is narrow, jitter dominates
- Same-zone UUID fix backfired: gap collapsed to 7ms, paradox exploded to 44%
- **Trimmed mean** (drop worst sample) reduces paradox from 26% to 18%



Paradox Rate Is Driven by the Cached-Uncached Gap



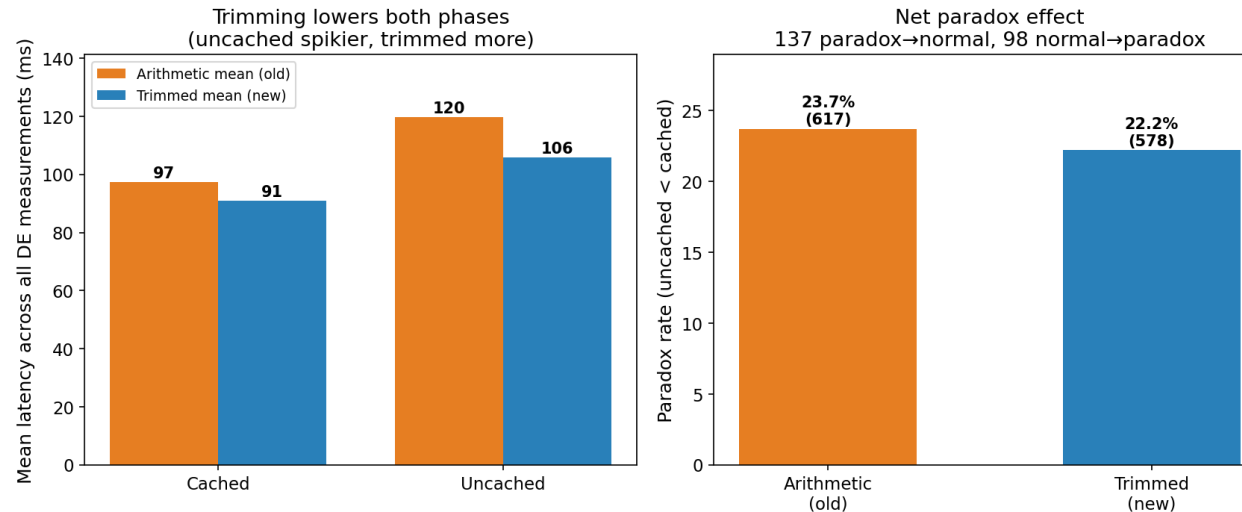
- $r = -0.72$ across locations
- DE: narrowest gap (10.9ms), 30.7% paradox
- PL: widest (28.5ms), 11.0%



- Trimmed mean: paradox flat ~18% across 11 locations
- Old 64% drop was an arithmetic-mean artifact
- Genuine priming ~14ms

Germany: Arithmetic Mean vs. Trimmed Mean on 2,600 Identical Samples

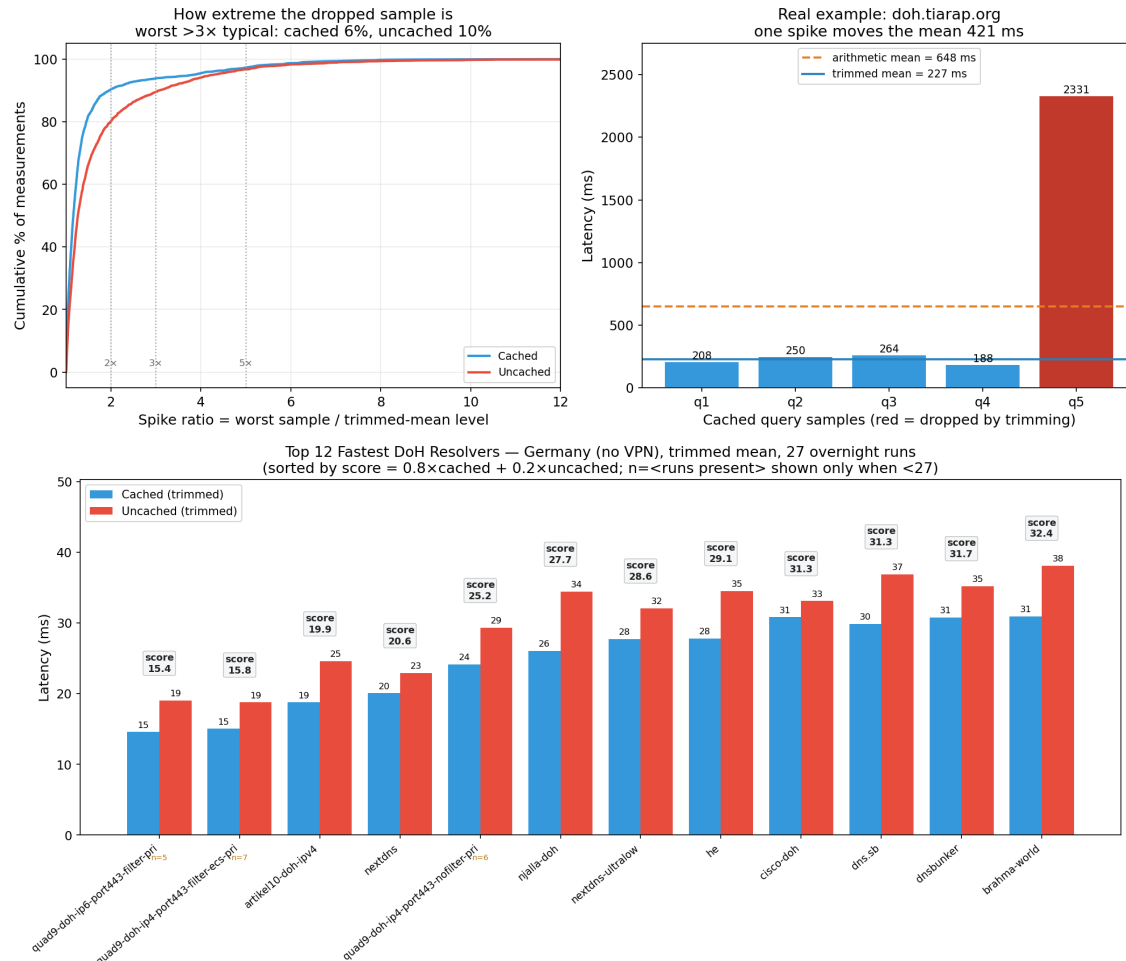
What the trimmed-mean update got us — Germany (no VPN), 27 overnight runs, n=2600 measurements on identical raw samples



- 27 overnight runs, local German WiFi, no VPN: 2,600 per-resolver measurements, both estimators on identical raw data
- Trimming removes symmetrically: 137 paradox to normal, 98 normal to paradox
- Worst single-sample inflation: 421ms cached, 506ms uncached. A 648ms arithmetic mean becomes 227ms trimmed

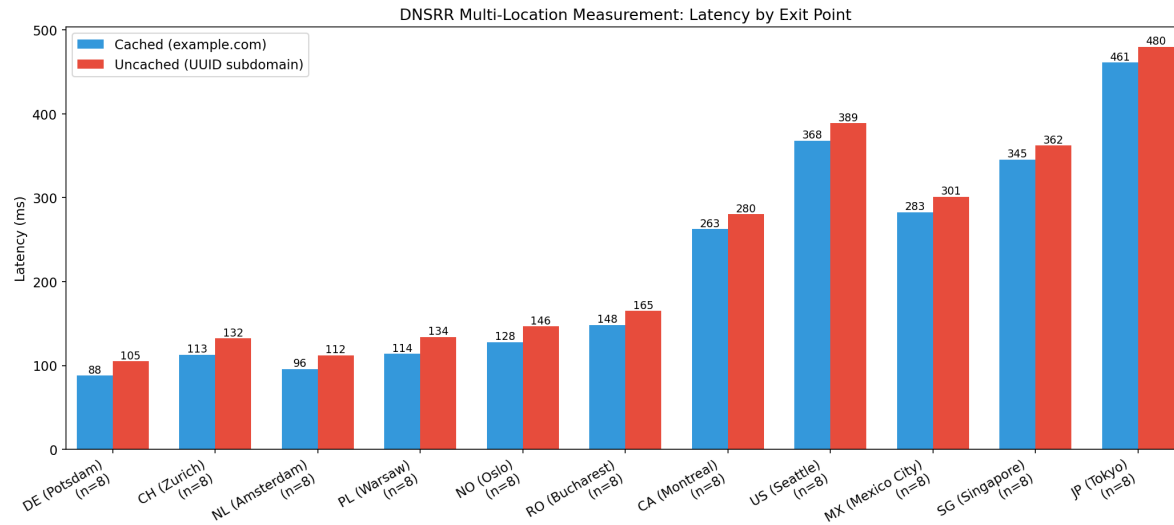
Spike Robustness and the German Resolver Ranking

Why trimming is needed — a single jitter spike dwarfs the typical latency (Germany overnight)

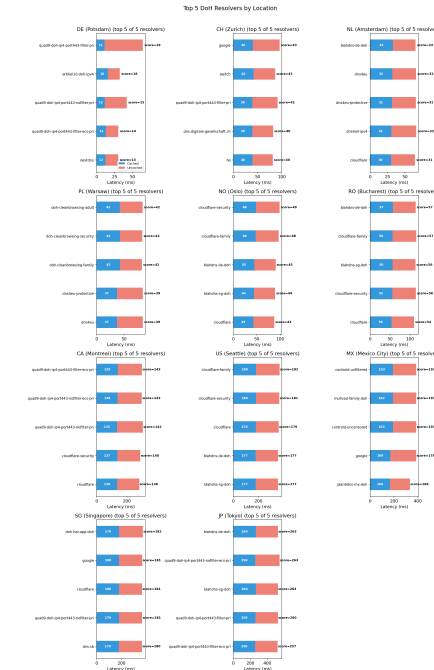


- doh.tiarap.org: [208, 250, 264, 188, **2331**]ms → arithmetic 648ms, trimmed 227ms
- Quad9 variants ~15-20ms cached but appear in only 5-7 of 27 runs (HTTP/3 failures)
- artike11 and nextdns present in all 27 runs: most dependable

The Fastest Resolver Depends on Where You Are

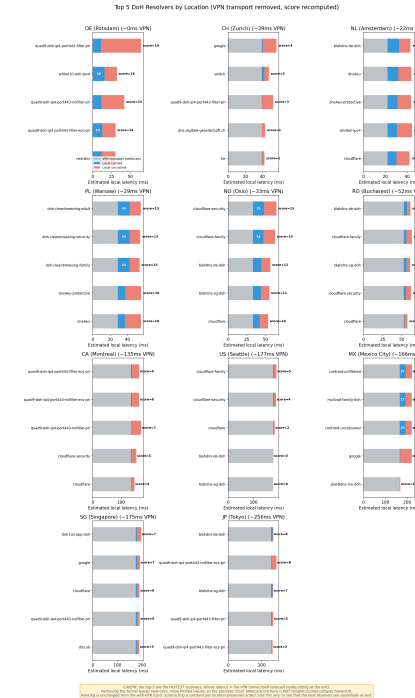
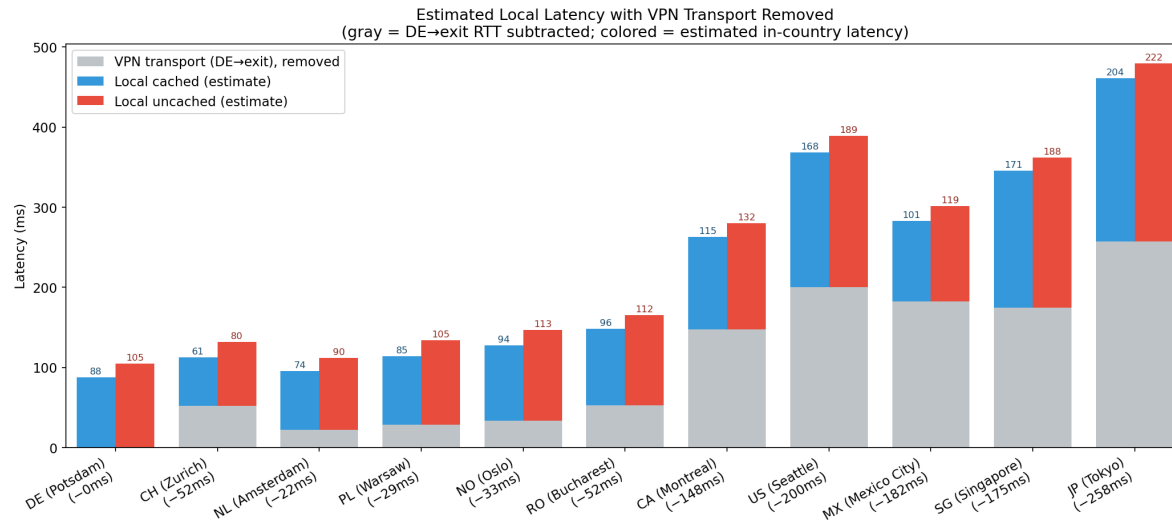


- 51 runs, 11 locations: DE (WiFi) + 10 VPN exits worldwide (ProtonVPN)
- dnsforge.de (12.6ms, DE) appears in no other top 5



- dns4eu dominates Europe, Quad9 leads N. America and Asia
- IPv6-only resolvers fail through IPv4-only VPN

VPN Transport Removal: Estimated Native In-Country Latency



- Tunnel RTT via TCP handshake timing to each VPN exit IP
- US 356ms to ~168ms, JP 472ms to ~204ms
- “US and Japan are slow” is a measurement-path artifact

What the Recommender Can and Cannot Do



- **Rankings, not absolute latencies:** Browser Fetch includes TCP/TLS/HTTP2 overhead (60–150ms). The DNS latency shown in the browser is the full HTTP round-trip, not pure DNS resolution time. The recommender produces defensible relative rankings, not 100% accurate absolute latencies
- **Single-user, single-location snapshot:** No diurnal tracking, no multi-device comparison, no longitudinal data per user
- **WiFi adds jitter:** 3–8ms L2 jitter inflates paradox rates vs. wired Ethernet. Ranking is robust but latency bars bounce
- **Shallow resolver metadata:** Three self-declared stamp bits, no third-party audit
- **More measurements planned:** Diurnal cycle, wired-Ethernet control, and native in-country runs before final presentation

Next Steps: From Speed Ranking to Personalized Recommender



UI Redesign

- Clean single-page design, simple + expert mode



Filter-Based Recommendation

- Enrich profiles: DNSSEC, filtering category, logging policy, jurisdiction
- Users set hard constraints, survivors ranked by measured speed



More Measurements

- 24h diurnal cycle, wired-Ethernet control, server-side calibration probe
- Community-driven measurements: invite users worldwide to run the tool natively



Canary Zone Verification

- Split-zone architecture: speed zone stays on Cloudflare Anycast for geographic fairness
- New canary sub-zone (`*.verify.diic-hpi.org`) delegated to Hetzner VM
- Browser fires one asynchronous canary query alongside the 5 speed queries
- Backend tails local DNS logs: instant verification, no 90s Cloudflare polling

Longer-Term Ideas

- Longitudinal stability, multi-browser comparison, browser extension

- [1] Bottger, T., Cuadrado, F., Antichi, G., Fernandes, E. L., Tyson, G., Castro, I., & Uhlig, S. (2019). An empirical study of the cost of DNS-over-HTTPS. *Proc. IMC '19*, 15–28.
- [2] Cloudflare. (2026). *Analytics, GraphQL API, Sampling*. Cloudflare Docs.
- [3] DNSCrypt. (2026). *DNS Stamps specification*.
- [4] Garcia, S., Hynek, K., Vekshin, D., Cejka, T., & Wasicek, A. (2021). Large scale measurement on the adoption of encrypted DNS. *arXiv:2107.04436*.
- [5] Hoffman, P., & McManus, P. (2018). *DNS Queries over HTTPS (DoH)* (RFC 8484). IETF.
- [6] IETF. (2024). *The DNS Stamps Specification* (draft-denis-dns-stamps-01).
- [7] Jerabek, K., Rysavy, O., & Burgetova, I. (2023). Analysis of well-known DNS over HTTPS resolvers. *Proc. IEEE CCWC*, 516–524.
- [8] Jung, J., Sit, E., Balakrishnan, H., & Morris, R. (2003). DNS performance and the effectiveness of caching. *IEEE/ACM Trans. Networking*, 10(5), 589–603.
- [9] Li, R., Jia, X., Zhang, Z., Shao, J., Lu, R., Lin, J., Jia, X., & Wei, G. (2023). A longitudinal and comprehensive measurement of DNS strict privacy. *IEEE/ACM Trans. Networking*, 31(6), 2793–2808.
- [10] Nosyk, Y., Korczynski, M., Lone, Q., Zhauniarovich, Y., & Duda, A. (2026). ADoX deployment in the wild. *RIPE Labs*.
- [11] Sharma, R., & Feamster, N. (2025). Measuring the availability and response times of public encrypted DNS resolvers. *arXiv:2208.04999v2*.
- [12] Verma, A. (2025). Encrypted DNS & privacy-preserving name resolution [Course project, Washington University in St. Louis].
- [13] Wahle, J., Ruas, T., Mohammad, S. M., Meuschke, N., & Gipp, B. (2023). AI usage cards: Responsibly reporting AI-generated content. *Proc. JCDL*, 282–284.
- [14] WhatWG. (2024). *Fetch Standard, 5.4. Response classes*.

AI Usage Disclosure



AI Usage Card			
PROJECT DETAILS	PROJECT NAME	DOMAIN	KEY APPLICATION
CONTACT(S)	NAME(S)	EMAIL(S)	
MODEL(S)	MODEL NAME(S)		
LITERATURE REVIEW	FINDING LITERATURE	FINDING EXAMPLES FROM KNOWN LITERATURE OR ADDING LITERATURE FOR EXISTING STATEMENTS	COMPARING LITERATURE
WRITING	GENERATING NEW TEXT BASED ON INSTRUCTIONS	ASSISTING IN IMPROVING OWN CONTENT OR PARAPHRASING RELATED WORK	PUTTING OTHER WORKS IN PERSPECTIVE
CODING	GENERATING NEW CODE BASED ON DESCRIPTIONS OR EXISTING CODE	REFACTORING AND OPTIMIZING EXISTING CODE	COMPARING ASPECTS OF EXISTING CODE
ETHICS	WHY DID WE USE AI FOR THIS PROJECT?	WHAT STEPS ARE WE TAKING TO MITIGATE ERRORS OF AI?	WHAT STEPS ARE WE TAKING TO MINIMIZE THE CHANCE OF HARM OR INAPPROPRIATE USE OF AI?
THE CORRESPONDING AUTHORS VERIFY AND AGREE WITH THE MODIFICATIONS OR GENERATIONS OF THEIR USED AI-GENERATED CONTENT			
AI Usage Card v2.6		https://ai-cards.org	PDF

Thank You



Guiding Questions for Discussion

- Can we verify opaque DoH resolvers from inside the browser sandbox, or do we need a server-side witness?
- How should a recommender trade off measured speed against unverifiable self-declared privacy claims?
- What matters most to you: raw speed, privacy, filtering policy, or jurisdiction?



Try the tool: **dns.diic-hpi.org**

Login: **dnsrr / dnsrr**

Appendix: Tier-2 Cloudflare GraphQL Verification



How it works:

- Each uncached query uses a unique UUID subdomain (<uuid>.diic-hpi.org)
- Backend polls Cloudflare GraphQL dnsAnalyticsAdaptive dataset
- Checks if the UUID appeared in authoritative DNS query logs
- Exponential backoff: 10s → 8s → 16s → 32s (90 s budget)
- If found: resolver verified. If not found after 90 s: marked unverified

Key limitation: dnsAnalyticsAdaptive uses **adaptive sampling** – coverage drops as data volume increases. Cloudflare docs state it “cannot be directly controlled.” Legitimate resolvers may not appear in the analytics. This is why Tier-1 CORS verification is architecturally necessary, not a convenience.

GraphQL query:

```
query ($zoneTag: String!,
      $queryName: String!,
      $since: String!) {
  viewer {
    zones(filter:{zoneTag:$zoneTag}){
      dnsAnalyticsAdaptive(
        filter: {
          datetime_geq: $since
          queryName: $queryName
        }
        limit: 10
      ) {
        queryName
        responseCode
        queryType
        datetime
      }
    }
  }
}
```